SIMATS ENGINEERING
CO4-ASSESSMENT TOOL -11
Saveetha Institute of Medical and Technical Sciences
Chennai- 602105

Student Name: TALARI VISHNUVARDHN **Reg. No.: 192572091**

Course Code: **CSA1511** **Slot:** B

Course Name: **CLOUD COMPUTING AND BIG DATA ANALYTICS**

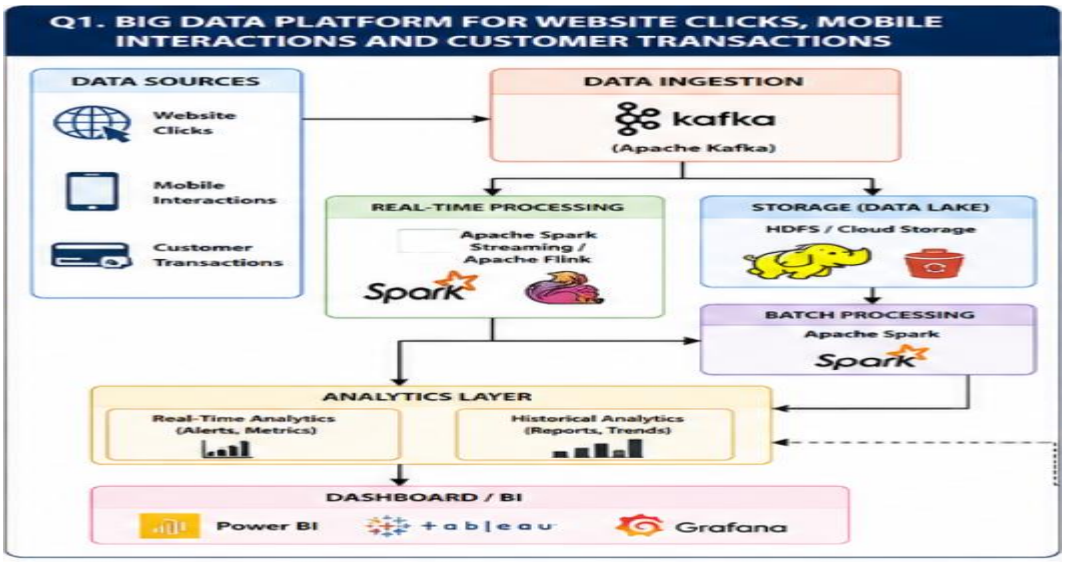
Course Faculty: Dr. Hemavathi R

QUESTION 1: BIG DATA PLATFORM FOR WEBSITE CLICKS, MOBILE INTERACTIONS AND CUSTOMER TRANSACTIONS

Answer

A big data platform that collects billions of website clicks, mobile interactions, and customer transactions every day requires a distributed architecture. The system must support high-volume data ingestion, distributed storage, batch processing, real-time processing, scalability, fault tolerance, and business intelligence. The probable components and design decisions are described below.

Diagram



1. Distributed Storage System

The platform would probably use a distributed storage system such as Hadoop Distributed File System (HDFS) or cloud-based data-lake storage.

HDFS divides large datasets into blocks and distributes them across multiple machines. Data replication is used to protect the data from hardware failures.

The storage layer can contain:

- Website click data
- Mobile interaction data
- Customer transaction records
- Application logs
- Historical analytical data

Distributed storage provides high capacity, scalability, and fault tolerance.

2. Data Ingestion Framework

Apache Kafka would be a suitable data-ingestion framework for this platform.

Website clicks, mobile interactions, and transactions can be converted into events and sent to Kafka. Different Kafka topics can be created for different types of data.

For example:

- Website clicks
- Mobile interactions
- Customer transactions

Kafka can handle continuous and high-volume data streams efficiently.

3. Structured and Semi-Structured Data

The platform handles both structured and semi-structured data.

Structured data includes:

- Customer records
- Transaction records
- Product information
- Payment information

Semi-structured data includes:

- JSON files

- Clickstream data
- Application logs
- Mobile events

Structured data can be stored in relational databases, while semi-structured data can be stored in a data lake or NoSQL database.

4. Indexing and Partitioning

Large datasets need to be partitioned so that queries can be executed efficiently.

Data can be partitioned according to:

- Date
- Region
- Customer
- Event type
- Product

For example, clickstream data can be partitioned by year, month, day, and region.

Partitioning reduces the amount of data that must be scanned during analysis.

5. Batch Processing

Historical data can be processed using Apache Spark or Hadoop MapReduce.

Batch processing can perform:

- Data cleaning
- Data transformation
- Aggregation
- Customer analysis
- Transaction analysis
- Trend analysis

Apache Spark can process large datasets in parallel across multiple machines.

6. Real-Time Stream Processing

Real-time data can be processed using Apache Spark Streaming, Spark Structured Streaming, Apache Flink, or Kafka Streams.

The processing flow can be:

Website/Mobile Applications → Kafka → Stream Processing → Real-Time Analytics

This allows the organization to identify important events immediately**.**

7. Scalability

The system can use horizontal scalability.

Instead of depending on one powerful server, multiple machines can work together as a cluster. When the amount of data increases, additional nodes can be added.

Kafka partitions can also distribute incoming events across multiple consumers.

8. Fault Tolerance

Fault tolerance can be provided using:

- Data replication
- Kafka replication
- Distributed storage
- Checkpointing
- Backups
- Automatic recovery

If one machine fails, the remaining machines can continue processing the workload.

9. Analytics Layer

The analytics layer can combine real-time and historical information.

Real-time analytics can be used for:

- Live customer activity
- Website traffic

- Transaction monitoring
- Event detection

Historical analytics can be used for:

- Business reports
- Customer trends
- Sales analysis
- Long-term performance analysis

10. Dashboard and Business Intelligence

The processed data can be connected to business intelligence tools such as Power BI, Tableau, or Grafana.

Dashboards can display:

- Website traffic
- Customer activity
- Transaction volume
- Regional performance
- Product performance
- Business trends

Conclusion

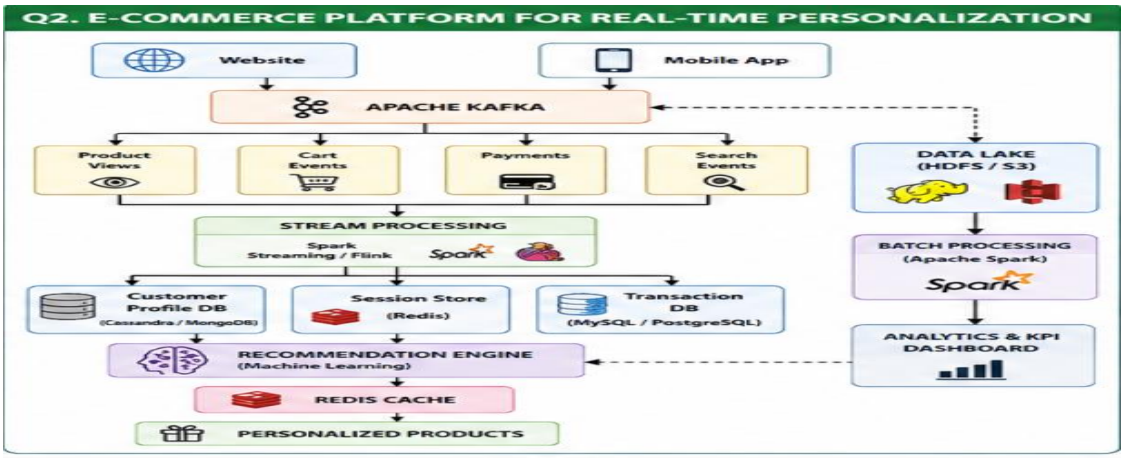
The probable big data architecture consists of distributed storage, Apache Kafka for data ingestion, Apache Spark or Flink for processing, partitioning for scalability, replication for fault tolerance, and BI dashboards for visualization. This architecture can efficiently handle billions of website clicks, mobile interactions, and customer transactions while supporting both real-time and historical analytics.

QUESTION 2: E-COMMERCE PLATFORM FOR REAL-TIME PERSONALIZATION

Answer

An e-commerce platform generates huge amounts of data from product views, cart additions, abandoned checkouts, payments, customer sessions, and browsing activity. A big data architecture can process this information in real time to understand customer behavior, generate personalized recommendations, and monitor business performance.

Diagram



1. Event Streaming

Apache Kafka would be a suitable event-streaming platform.

Customer activities such as product views, cart additions, checkout events, payment events, and searches can be published as events.

Different Kafka topics can be used for:

- Product views
- Cart events
- Checkout events
- Payment events
- Search events

This provides reliable and high-throughput event processing.

2. Customer Profiling Database

The platform can use both relational and NoSQL databases.

Relational databases can store structured information such as:

- Customer ID
- Order ID
- Product ID
- Price
- Payment status

NoSQL databases can store rapidly changing customer activity, browsing history, and customer profiles.

3. Session Data

Session data represents the activities performed by a customer during a particular browsing session.

For example:

Customer → Product View → Product Search → Add to Cart → Checkout

The system can use Customer ID and Session ID to connect these events.

This allows the platform to understand the customer's current behavior.

4. Real-Time Synchronization

Customer events can be synchronized in real time using Kafka and a stream-processing engine.

The flow is:

Customer Activity → Kafka → Stream Processing → Customer Profile Update

When a customer performs an activity, the customer profile can be updated immediately.

5. Recommendation Workflow

The recommendation workflow can include the following steps:

1. Collect customer activity.
2. Clean and transform the data.
3. Create customer profiles.

4. Extract useful features.
5. Train machine learning models.
6. Generate recommendations.
7. Store recommendations.
8. Display personalized products to the customer.

6. Machine Learning

Machine learning can be used for:

- Product recommendation
- Customer segmentation
- Purchase prediction
- Customer behavior analysis
- Churn prediction
- Demand prediction

Recommendation techniques such as collaborative filtering and content-based recommendation can be used.

7. Caching System

Redis can be used as a caching system.

Frequently accessed information such as:

- Product information
- Customer sessions
- Recommendations
- Shopping cart information

can be stored in the cache.

Caching reduces database load and improves response time.

8. Distributed Query Engine

Distributed query engines such as Apache Spark SQL, Presto, or Trino can process large datasets.

They can combine information from:

- Order records
- Clickstream data
- Customer profiles
- Product information
- Payment information

9. Merging Structured and Semi-Structured Data

Order information is generally structured, while browsing and clickstream information is often semi-structured.

These datasets can be combined using common fields such as:

- Customer ID
- Product ID
- Session ID
- Timestamp

This provides a complete view of customer behavior.

10. Scalability

The platform can use horizontal scaling by adding more servers.

Kafka partitions can distribute events across multiple consumers. Distributed databases can distribute customer data across multiple nodes, while Spark can distribute processing tasks across a cluster.

11. Fault Tolerance

Fault tolerance can be achieved using:

- Kafka replication
- Database replication
- Distributed storage
- Checkpointing

- Backups
- Automatic recovery

These mechanisms help the system continue operating even when individual components fail.

12. Dashboard and KPI Monitoring

The dashboard can display important KPIs such as:

- Total sales
- Conversion rate
- Cart abandonment rate
- Number of visitors
- Average order value
- Popular products
- Customer retention

Conclusion

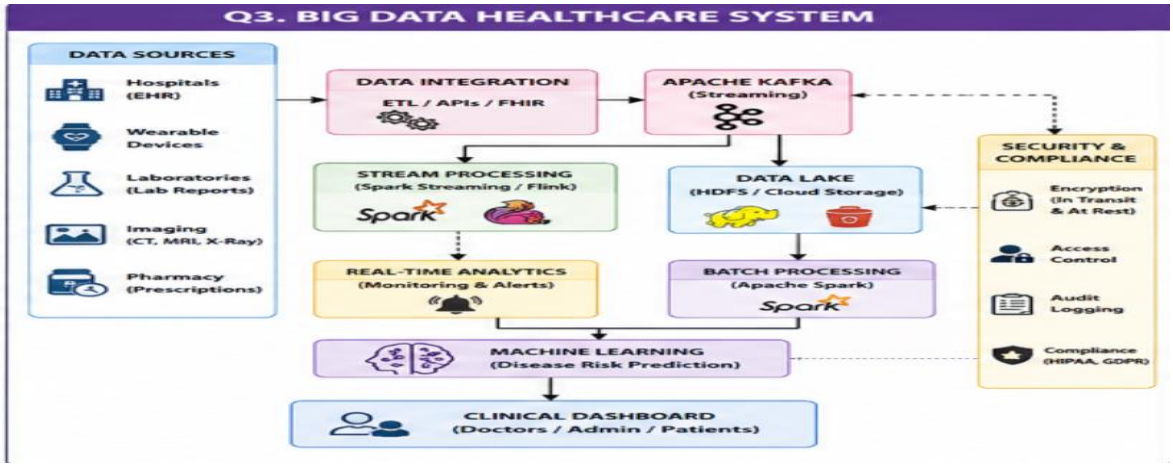
The probable e-commerce big data architecture combines Apache Kafka, stream processing, customer databases, Redis caching, distributed query engines, machine learning, and business intelligence dashboards. This architecture enables real-time customer profiling, personalized recommendations, scalable processing, and continuous KPI monitoring.

QUESTION 3: BIG DATA HEALTHCARE SYSTEM

Answer

A healthcare big data system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Since healthcare information is available in structured, semi-structured, and unstructured forms and is highly sensitive, the system requires hybrid storage, data integration, real-time processing, machine learning, encryption, privacy protection, access control, and auditing.

Diagram



1. Healthcare Data Sources

The healthcare platform can collect data from:

- Hospitals
- Laboratories
- Wearable devices
- Patient records
- Diagnostic imaging systems
- Prescription systems
- Insurance systems

These sources produce different types of healthcare information**.**

2. Hybrid Database Architecture

A hybrid storage architecture is suitable because different types of data require different storage systems.

Relational databases can store:

- Patient information
- Prescriptions
- Appointments
- Laboratory results

NoSQL databases can store:

- Sensor readings
- Semi-structured healthcare events
- Patient activity data

Distributed storage or a data lake can store:

- X-ray images
- CT scans
- MRI images
- Medical documents
- Large historical datasets

3. Data Integration

Healthcare information may be distributed across hospitals, laboratories, clinics, and insurance systems.

ETL pipelines, APIs, and interoperability standards can be used to integrate these systems.

Healthcare interoperability standards such as HL7 and FHIR can help different healthcare systems exchange information in a standardized manner.

4. Data Ingestion

Apache Kafka can be used to collect continuous wearable sensor data and other real-time events.

The basic flow is:

Wearable Devices → Kafka → Stream Processing → Healthcare Analytics

Historical healthcare records can be loaded into the data lake using batch ETL pipelines.

5. Real-Time Stream Processing

Real-time processing can be performed using Apache Spark Streaming, Spark Structured Streaming, or Apache Flink.

The system can continuously process wearable sensor readings and other real-time events.

This supports real-time monitoring and alert generation.

6. Machine Learning

Machine learning can be used for:

- Disease prediction
- Risk prediction
- Patient monitoring
- Medical image analysis
- Readmission prediction
- Population health analysis

The general process is:

Healthcare Data → Data Preprocessing → Feature Extraction → Machine Learning Model → Prediction

7. Privacy and Security

Healthcare information is sensitive and must be protected.

Security mechanisms include:

- Authentication
- Authorization
- Role-based access control
- Encryption
- Data masking
- Audit logging
- Secure communication

Only authorized users should be allowed to access patient information.

8. Encryption

Encryption should be applied to data both at rest and in transit.

Data at rest means information stored in databases, data lakes, and storage systems.

Data in transit means information transferred between hospitals, applications, devices, and servers.

9. Compliance and Governance

The platform should follow applicable healthcare privacy, security, and data-protection requirements.

Governance policies should define:

- Who can access data
- What data can be accessed
- How data is stored
- How data is shared
- How access is recorded

10. Clinical Decision Support

The analytics system can process patient information and provide useful risk information to authorized healthcare professionals.

The workflow can be:

Patient Data → Analytics → Risk Prediction → Clinical Decision Support

This can assist healthcare professionals in making informed decisions.

11. Population Health Analytics

Distributed processing frameworks such as Apache Spark can analyze large numbers of patient records.

Population-level analysis can identify:

- Disease trends
- High-risk populations
- Regional health patterns

- Treatment patterns
- Healthcare requirements

Conclusion

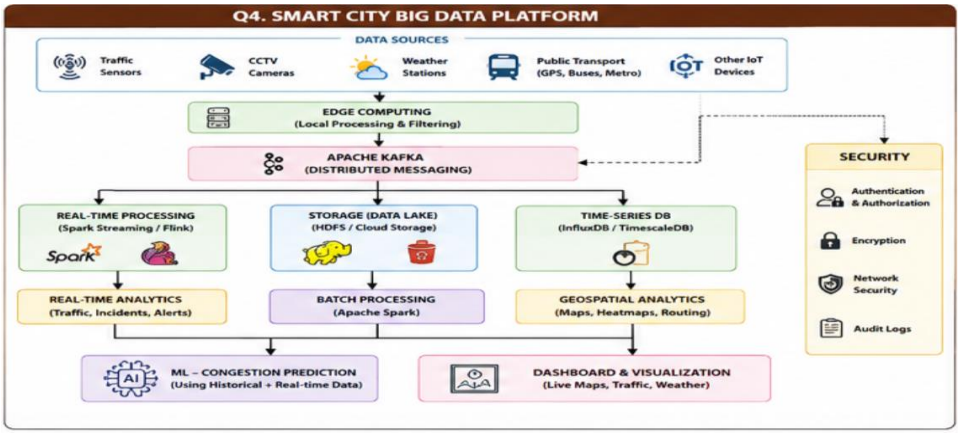
The probable healthcare big data system uses hybrid databases, distributed storage, Kafka, stream processing, Apache Spark, machine learning, encryption, access control, and auditing. This architecture supports real-time monitoring, disease prediction, clinical decision support, and population health analysis while protecting sensitive healthcare information.

QUESTION 4: SMART CITY BIG DATA PLATFORM

Answer

A smart city platform continuously collects data from traffic sensors, CCTV metadata, weather stations, public transportation, and other IoT devices. Because the data is generated continuously, the platform requires edge computing, distributed messaging, centralized storage, real-time processing, geospatial analytics, time-series databases, machine learning, security, and visualization.

Diagram



1. Data Sources

The smart city platform can collect information from:

- Traffic sensors
- CCTV systems
- Weather stations
- Public transportation
- GPS devices
- Other IoT devices

These sources continuously generate large volumes of data.

2. Edge Computing

Edge computing allows data to be processed close to the location where it is generated.

For example:

Traffic Sensor → Edge Device → Local Processing → Central Platform

Edge computing reduces network latency and decreases the amount of data that must be transmitted to the central system.

3. Distributed Messaging System

Apache Kafka can be used as a distributed messaging system.

Different topics can be created for:

- Traffic
- Weather
- Transportation
- CCTV
- IoT sensors

Kafka partitions allow multiple consumers to process events simultaneously.

4. Centralized Storage

Historical smart-city data can be stored in HDFS or cloud-based data-lake storage.

The storage system can retain:

- Traffic history
- Weather history
- Transportation data
- Sensor information
- CCTV metadata

Historical data can later be used for urban planning and trend analysis.

5. Real-Time Stream Processing

Apache Spark Streaming, Spark Structured Streaming, or Apache Flink can process real-time data.

The processing flow is:

Traffic Sensors → Kafka → Stream Processing → Real-Time Analytics

The system can detect traffic congestion, transportation delays, and other events.

6. Geospatial Analytics

Geospatial analytics is important because smart-city data is associated with physical locations.

It can be used to identify:

- Traffic congestion locations
- Vehicle routes
- Road usage
- Accident-prone locations
- Public transport routes

Location-aware databases can support efficient geographical queries.

7. Time-Series Database

Traffic sensors and other IoT devices generate data continuously over time.

A time-series database can store:

- Sensor ID
- Timestamp
- Vehicle count
- Traffic speed
- Temperature
- Other sensor measurements

This makes it possible to analyze changes over time.

8. Congestion Prediction

Machine learning can be used to predict traffic congestion.

The model can use:

- Historical traffic data
- Real-time traffic data
- Weather conditions
- Time of day
- Location
- Public events

The process is:

Historical Data + Real-Time Data → Machine Learning → Congestion Prediction

9. Batch Analytics

Historical data can be processed using Apache Spark.

Batch processing can identify:

- Long-term traffic trends
- Transportation patterns
- Road utilization
- Weather effects
- Infrastructure requirements

10. Real-Time Analytics

Real-time analytics can provide immediate information about:

- Current traffic conditions
- Congestion
- Public transport delays
- Weather conditions
- Sensor failures

11. Security and Access Control

Smart-city platforms require strong security because they may contain important infrastructure information.

Security measures include:

- Authentication
- Authorization
- Encryption
- Role-based access control
- Network security
- Audit logs

12. Visualization

Visualization systems can provide live operational intelligence.

Dashboards can display:

- Live traffic maps
- Congestion levels
- Weather conditions
- Public transportation status
- Sensor status

These dashboards help city authorities monitor and manage urban systems.

Conclusion

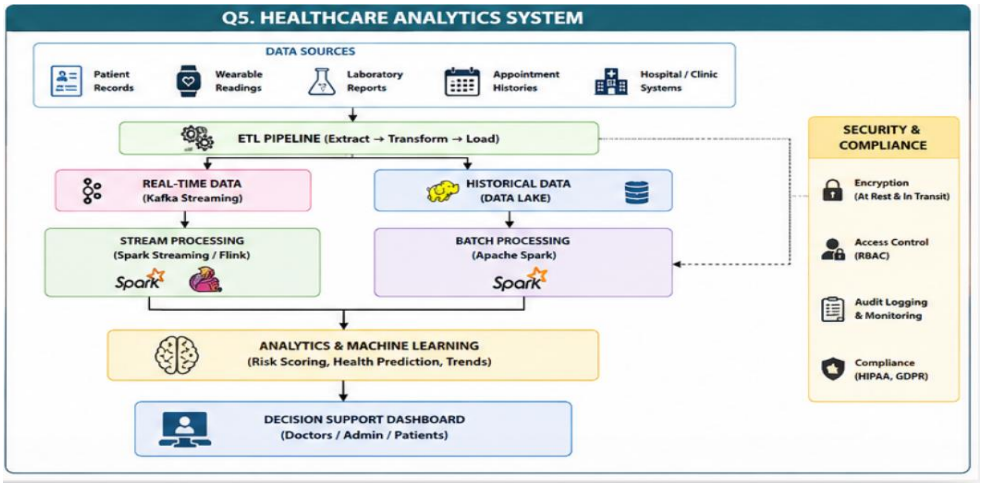
The probable smart-city architecture combines edge computing, Apache Kafka, distributed storage, stream processing, time-series databases, geospatial analytics, machine learning, security, and visualization. The architecture supports both real-time operational monitoring and historical analytics for urban planning.

QUESTION 5: HEALTHCARE ANALYTICS SYSTEM

Answer

A healthcare analytics system integrates patient records, wearable device readings, laboratory reports, and appointment histories. Since healthcare information is sensitive and comes from multiple sources, the platform requires hybrid storage, ETL pipelines, interoperability standards, real-time monitoring, distributed processing, machine learning, encryption, access control, compliance, and auditing.

Diagram



1. Data Sources

The system collects data from:

- Patient records
- Wearable devices
- Laboratory systems
- Appointment systems
- Hospitals
- Clinics

The collected information can be structured, semi-structured, or unstructured.

2. Hybrid Storage Architecture

Different storage systems can be used for different types of healthcare information.

Relational databases can store:

- Patient records
- Appointment histories
- Laboratory values
- Administrative information

NoSQL or time-series databases can store:

- Wearable readings
- Sensor events
- Frequently changing health information

Data lakes or distributed object storage can store:

- Medical documents
- Large historical datasets
- Unstructured information

3. ETL Pipeline

The healthcare platform can use an ETL pipeline.

ETL stands for:

Extract → Transform → Load

Extract

Data is collected from hospitals, clinics, laboratories, wearable devices, and appointment systems.

Transform

The collected data is:

- Cleaned
- Validated
- Standardized
- Transformed

- Prepared for analysis

Load

The processed data is loaded into the appropriate database or data lake.

4. Interoperability

Different hospitals and clinics may use different systems.

Healthcare interoperability standards such as HL7 and FHIR can help exchange healthcare information between systems.

This allows patient information to move between authorized healthcare organizations in a standardized format.

5. Real-Time Monitoring

Wearable devices continuously generate health-related readings.

The probable real-time architecture is:

Wearable Device → Kafka → Stream Processing → Real-Time Monitoring → Dashboard/Alert

Stream-processing technologies such as Apache Spark Streaming or Apache Flink can process incoming data.

6. Predictive Health Analytics

Machine learning can analyze patient information to identify potential health risks.

Applications include:

- Disease risk prediction
- Patient risk scoring
- Readmission prediction
- Treatment outcome analysis
- Health deterioration detection

7. Distributed Processing

Large healthcare datasets can be processed using Apache Spark.

The data can be divided into smaller tasks and processed in parallel across multiple machines.

This improves processing speed and supports population-level analysis.

8. Population-Level Analysis

Large datasets can be analyzed to identify:

- Disease trends
- Regional health patterns
- Treatment patterns
- High-risk groups
- Healthcare resource requirements

This information can support healthcare planning and decision-making.

9. Encryption

Sensitive healthcare information should be encrypted.

Encryption at Rest: Protects data stored in databases and data lakes.

Encryption in Transit: Protects information transferred between devices, hospitals, databases, and applications.

10. Access Control

Role-based access control can restrict access to healthcare information.

Users should receive only the access required for their responsibilities.

Authentication and authorization should be implemented before sensitive information is accessed.

11. Compliance and Audit

The system should maintain audit records of important activities such as:

- User login
- Data access
- Data modification
- Data sharing
- Administrative operations

Audit logs help detect unauthorized access and support data governance and compliance.

12. Machine Learning for Risk Scoring

The machine learning workflow can be:

Patient Records + Wearable Data + Laboratory Reports + Appointment History

↓

Data Preprocessing

↓

Feature Engineering

↓

Machine Learning Model

↓

Risk Score

↓

Decision Support

Machine learning can help identify patients who may require additional attention.

13. Treatment and Decision Support

Analytics can provide risk information and predictive insights to authorized healthcare professionals.

The system can support evaluation of:

- Patient risk
- Treatment response
- Follow-up requirements
- Potential health deterioration

The final medical decision should be made by qualified healthcare professionals.

Conclusion

The healthcare analytics system combines hybrid storage, ETL pipelines, interoperability standards, Apache Kafka, stream processing, Apache Spark, machine learning, encryption, access control, compliance, and auditing.